Regular Expressions (RegEx)

Regular Expressions (RegEx) are patterns used for matching character combinations in strings. They are widely used for searching, replacing, validating input formats, and text processing.

Python provides the `re` module for working with regular expressions.

---

1. Why Use Regular Expressions?

Regular expressions are used to simplify text processing tasks that would otherwise require lengthy conditional logic. They are useful for:

- Validating input data (e.g., email, phone numbers, passwords)
- Searching and extracting specific information from text files and logs
- Replacing patterns in strings (e.g., replacing multiple spaces with a single space)
- Splitting text based on specific delimiters

---

2. Importing the `re` Module

Before using RegEx, import Python's built-in `re` module:

```python
import re
```

---

3. Writing a Basic Regular Expression

A basic RegEx function consists of:

1. A pattern (the search condition)
2. A string (where to search)
3. A function to process the match (`search()`, `match()`, `findall()`, etc.)

Example: Extracting Numbers from Text

```python
import re

pattern = r"\d+"   # Match one or more digits

text = "My phone number is 9876543210"

match = re.search(pattern, text)  # Search for the pattern

if match:

    print("Match found:", match.group())

else:

    print("No match found")
```

Output:

```
Match found: 9876543210
```

4. Common Python RegEx Functions

Function	Description
`re.match()`	Checks if the pattern matches **only at the start** of the string.
`re.search()`	Searches for the **first** occurrence of the pattern anywhere in the string.
`re.findall()`	Returns **all** matches in a list.

`re.finditer()`	Returns an **iterator** of match objects.
`re.fullmatch()`	Checks if the **entire string** matches the pattern.
`re.sub()`	Replaces **matches** with another string.
`re.split()`	Splits a string at each match.

5. Detailed Examples of Each RegEx Function

1. `re.match()` → Matches at the Start

```python
import re

result = re.match(r"Hello", "Hello World")

print(result.group() if result else "No match")
```

Output:

```
Hello
```

---

2. `re.search()` → Finds First Match Anywhere

```python
import re

result = re.search(r"\d+", "My age is 25 and my ID is 1001")

print(result.group() if result else "No match")
```

Output:

```
Python
25
```

---

3. `re.findall()` → Finds All Matches

```
Python
import re

result = re.findall(r"\d+", "I have 2 cats, 3 dogs, and 5
birds")

print(result)
```

Output:

```
Python
['2', '3', '5']
```

---

4. `re.finditer()` → Finds Matches as Objects

```
Python
import re

text = "Product IDs: 123, 456, 789"

matches = re.finditer(r"\d+", text)

for match in matches:

    print(f"Match: {match.group()} at position
{match.start()}")
```

Output:

```
Python
Match: 123 at position 12

Match: 456 at position 17

Match: 789 at position 22
```

---

5. `re.fullmatch()` → Checks Entire String

```
Python
import re

result = re.fullmatch(r"\d{10}", "9876543210")

print("Valid" if result else "Invalid")
```

Output:

```
Python
Valid
```

---

6. `re.sub()` → Replace Matches

```
Python
import re

text = "I love cats"

result = re.sub(r"cats", "dogs", text)

print(result)
```

Output:

```
I love dogs
```

---

7. `re.split()` → Split String at Matches

```python
import re

text = "apple, banana; orange - grape"

result = re.split(r"[,; -]+", text)

print(result)
```

Output:

```
['apple', 'banana', 'orange', 'grape']
```

6. Special Metacharacters in RegEx

Part 1: Basic Metacharacters

Metacharacter	Description	Example
`.`	Matches **any character** (except newline)	`h.t` → Matches `"hat"`, `"hot"`, `"hit"`
`^`	Matches **start** of a string	`^Hello` → Matches `"Hello World"`
`$`	Matches **end** of a string	`end$` → Matches `"The end"`
`*`	Matches **0 or more occurrences**	`ab*` → Matches `"a"`, `"ab"`, `"abb"`
`+`	Matches **1 or more occurrences**	`ab+` → Matches `"ab"`, `"abb"`
`?`	Matches **0 or 1 occurrence**	`colou?r` → Matches `"color"`, `"colour"`

Part 2: Advanced Metacharacters

Metacharacter	Description	Example
`[]`	Matches **any character inside brackets**	`[aeiou]` → Matches `"a"`, `"e"`, `"i"`, `"o"`, `"u"`
`\`	Escapes special characters	`\.` → Matches a **literal** `"."`
`{}`	Matches **exact number of occurrences**	`a{2}` → Matches `"aa"`
`()`	Groups patterns together	`(abc)+` → Matches `"abc"`, `"abcabc"`
`	`	`	OR operator
`\b`	Matches a word boundary	`\bcat\b` → Matches `"cat"` but not `"cats"`
`\d`	Matches any **digit** (0-9)	`\d+` → Matches `"123"`
`\D`	Matches any **non-digit**	`\D+` → Matches `"abc"`
`\s`	Matches any **whitespace** (space, tab, newline)	`\s+` → Matches `" "`

`\S`	Matches any **non-whitespace** character	`\S+` → Matches `"Hello"`
`\w`	Matches any **word character** (letters, digits, underscore)	`\w+` → Matches `"word123"`
`\W`	Matches any **non-word character**	`\W+` → Matches `"@#$%^"`